

FlyClient Guide

Sampling an authenticated proof-of-work history

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the sampling argument behind FlyClient. It assumes probability from the *Math Guide*, authenticated histories from the *Crypto Guide*, and work, chain selection and block-history commitments from the *Consensus Guide*. The endpoint is a verifier that checks a committed candidate history by sampling, together with an exact statement of what would still be needed to instantiate that verifier for Zcash. The sampling theorem is proved here. The mining assumption, variable-difficulty transition argument and network availability premise are kept separate from it.

Contents

1	The chain-verification contract	3
1.1	Three different questions	3
1.2	The assumption paid for by sampling	4
2	Work intervals and authenticated samples	4
2.1	From positions to work	4
2.2	A history inside every header	5
2.3	Difficulty transitions	5
3	Choosing the sampling distribution	6
3.1	Why the tip receives more samples	6
3.2	Normalisation and the inverse sampler	7
3.3	Optimality and its exact domain	7
3.4	Choosing the sample count	8
4	The interactive verifier	8
5	Fiat-Shamir and persistent verification	9
5.1	Binding the challenge	10
5.2	State retained by a returning client	10
6	Connecting the construction to Zcash	11

1 The chain-verification contract

The probability language used here—random choices, events, independence, and error bounds—is constructed in the *Math Guide*, §*Probability and computation*. That section also constructs real intervals, integration, probability densities, logarithms, and inverse sampling before they are used below. *Crypto Guide*, §*Security contracts and reductions*, supplies the computational meaning of a negligible failure probability and a security assumption.

A full node checks blocks and their transactions before using their work to select a chain. A resource-limited client would like to learn which history this selects without receiving every header. Receiving a short history commitment solves only half that problem: a hash binds a server to its answer, but does not make the answer valid.

FlyClient adds unpredictable tests of the committed history. A server first fixes a candidate head. The client then asks for selected earlier headers and checks both their work and their connection to that head. A candidate containing enough invalid work should fail a test. The distribution must catch forks near the tip as well as forks near genesis.

Our source is Bünz, Kiffer, Luu and Zamani, *FlyClient: Super-Light Clients for Cryptocurrencies*, revision 2020-08-20, especially Sections 4–6 and Appendix A. We use zero-based history indices consistently and separate the paper’s abstract construction from the Zcash history tree. The deployed tree’s exact encoding belongs to the *Consensus Guide*, §*Authenticated block history*; repeating it here would obscure the different object the sampling theorem assumes.

1.1 Three different questions

Let a candidate consist of headers

$$C = (B_0, \dots, B_n).$$

The head is B_n . The preceding n headers form its committed history. A header’s work is a positive integer w_i ; work and target are not synonyms. The *Consensus Guide*, §*Work, difficulty, and chain selection*, constructs Zcash’s work function and explains why a larger numerical target means an easier proof of work.

The client must distinguish three predicates, meaning conditions that can be true or false:

1. *Membership*: a particular header is at a particular position in the sequence committed by B_n .
2. *Header validity*: the candidate obeys its header-link, proof-of-work and difficulty rules.
3. *Ledger validity*: transactions, proofs, signatures, balances, nullifiers and all other state transitions are valid.

An authenticated path proves the first predicate. Sampling aims at the second, under explicit mining and commitment assumptions. Neither operation independently checks every transaction in the third.

A client accepting a transaction-inclusion proof therefore relies on the underlying consensus model’s honest full nodes to reject invalid ledger transitions. The transaction’s inclusion path does not itself prove that its spend was authorised. This distinction is the same one that separates a transaction digest from a valid transaction.

Definition 1.1 (Client outcome). The client receives candidate heads and their proofs from servers, at least one of which supplies an honest full node’s chain. It rejects malformed candidates and selects a surviving candidate of greatest verified claimed work. Security asks that the selected history agree with the honest chain away from its potentially unstable suffix.

The availability premise is substantive. A completely isolated client can receive a valid but old history and no evidence that a newer one exists. No sampling distribution manufactures a missing

honest view. Nor does agreement between two server responses prove that either server is independent or honest. The client checks the claimed objects; it does not replace validation by counting matching responses.

1.2 The assumption paid for by sampling

Suppose a competing history branches from the honest history at some point a . A server claiming at least the honest chain's work cannot ordinarily manufacture all of its fork with valid puzzles. Fly-Client formalises the resulting deficit through a bound on the fraction of post-fork work that can pass the sample verifier.

Assumption 1.2 (Bounded valid fork weight). Fix $0 < c < 1$ and a positive integer short-fork cutoff L . Except with probability $\varepsilon_{\text{mine}}$, every adversarial competing fork outside the short-fork exception has at most a c fraction of its claimed post-fork work in headers that pass the protocol's work, history and difficulty checks. The remainder is detectable invalid work.

This is a property of adversarial histories, not a definition of the adversary's hash rate. In particular, $c = 0.9$ does not mean that 90 percent of total mining power is adversarial. A theorem connecting hash-rate and network parameters to c, L must be established in the chosen chain model.

Short forks are excluded because mining has variance even under honest majority. The verifier checks a trailing region in full. In a constant-difficulty chain its length can be compared directly with L . In a variable-difficulty chain, a block-count cutoff and a work-fraction cutoff are different quantities. A concrete instantiation must make its fully checked suffix cover the short forks exempted by its mining assumption.

2 Work intervals and authenticated samples

The *Crypto Guide*, §*Authenticated trees and append-only histories*, supplies ordered inclusion proofs and append-only commitments. We now specify what their interface must expose to the sampler. This is additional metadata on the authenticated sequence, not a new security assumption about the hash function.

2.1 From positions to work

Define prefix work and total history work by

$$W_0 = 0, \quad W_i = \sum_{j=0}^{i-1} w_j, \quad W = W_n.$$

For $W > 0$, header B_i occupies the half-open interval

$$I_i = [W_i/W, W_{i+1}/W).$$

These intervals are disjoint and cover $[0, 1)$. A point x selects the unique header whose interval contains it. Half-open intervals fix boundary behaviour without double counting.

For equal work per block, $I_i = [i/n, (i+1)/n)$, so work sampling reduces to position sampling. For unequal work, a high-work block covers more of the domain. Sampling header positions uniformly would allow a server to hide its claimed work in a small number of artificially difficult headers.

Construction 2.1 (Authenticated weight lookup). Each tree node commits to its ordered children and their total weights. To locate a work coordinate $u = xW$, start with offset zero at the root. At a parent with left weight W_ℓ , descend left when $u < W_\ell$. Otherwise subtract W_ℓ , add W_ℓ to the offset,

and descend right. The proof supplies the sibling node at every step. The verifier recomputes each parent hash and weight, then checks that the final residual coordinate lies in the returned leaf’s interval.

The verifier checks lengths, child orientation, integer widths and overflow conditions before using supplied weights. Its final total must equal the total authenticated by the selected head. A server cannot choose a different leaf after seeing the sample without either changing the root or presenting an inconsistent path.

This claim is conditional on the node invariants. A hash commits to false metadata just as effectively as true metadata. Recomputing the visited ancestors checks the additions along that path; it does not recompute every unopened subtree. The chain-validation argument must account for malformed subtrees through the sample predicate.

2.2 A history inside every header

The abstract protocol requires header B_i to commit to the ordered prefix $(B_0, \dots, B_{i-1})$, not merely to its predecessor’s hash. Let that commitment be R_i . The head’s history root is R_n .

A sample proof has two jobs. First it proves that B_i appears under R_n . Second it reconstructs the commitment to the prefix strictly before B_i , and checks that this equals R_i inside the sampled header. Without the second check, a server could try to insert a previously mined header into a different committed history.

For an append-only forest, a prefix-recovery proof supplies the earlier forest peaks and the left siblings inside the peak containing leaf i . Together these partition the leaves before i into complete subtrees. Their authentication must connect them to the head’s root. Ordering these subtrees from left to right and applying the forest’s fixed root-folding rule reconstructs the prefix root. The route and leaf count determine their positions; supplied direction bits cannot redefine the leaf position.

The paper uses its recursively rooted MMR and the corresponding prefix-recovery procedure. Zcash uses the different bagging convention constructed in *Consensus Guide*, §*Authenticated block history*. The generic requirement is the same, but a proof for one root convention is not an encoding of a proof for the other.

Proposition 2.2 (History consistency of a sampled header). *Assume the ordered history commitment is binding. If a sampled header passes inclusion under R_n and its own R_i equals the recovered prefix root, then its committed earlier history is the same ordered prefix as that authenticated by the head, except with the commitment’s binding failure probability.*

Proof. Two different ordered prefixes giving the recovered root would violate binding. The equality with R_i therefore identifies one prefix in both commitments. Position binding additionally prevents interpreting the same opening as a different prefix length. $\square$

The proposition says nothing about the transactions inside that prefix. It prevents reuse of a header in an incompatible history; it does not turn the header into a proof of the entire preceding computation.

2.3 Difficulty transitions

Consider the paper’s epoch model. An epoch contains M blocks. A target T is replaced after elapsed time Δ by

$$T' = \text{clamp}(T\Delta f/M, T/\tau, \tau T), \quad \tau > 1,$$

where a round is the model’s unit of time and f is the intended blocks-per-round rate. Here $\text{clamp}(x, a, b) = \max(a, \min(x, b))$. Rounding and the permitted target range are part of a concrete rule. The clamp bounds each target change; it is essential to the model’s security analysis.

The paper’s difficulty tree aggregates a tuple

$$(h, w, t, D_{\text{start}}, D_{\text{next}}, n).$$

Here h authenticates the node, w is total difficulty weight, t is elapsed time, n is leaf count, and D denotes difficulty rather than the oppositely ordered proof-of-work target. A leaf has $n = 1$, its own weight and elapsed time, and the next difficulty prescribed by the rule. Joining left and right nodes adds w, t, n , inherits the left starting difficulty and right next difficulty, and authenticates both full tuples.

At an opened join, the verifier checks $D_{\text{next},\ell} = D_{\text{start},r}$. Within one epoch it checks constant difficulty and consistent aggregate weight. Across epochs it checks that the boundary difficulties, elapsed time and total weight admit legal clamped transitions.

For example, if a sequence of r epoch transitions begins at difficulty D , each transition changes difficulty by at most a factor τ . After j transitions its difficulty lies between $D\tau^{-j}$ and $D\tau^j$. An independently fixed ending difficulty adds the same constraints backwards. The maximum possible intermediate difficulty is bounded by the smaller of the forward and backward upper envelopes; the minimum is bounded by the larger lower envelope. Summing these envelopes bounds aggregate weight. Timing constraints must also be satisfied: a sequence of allowable difficulty levels alone does not show that the retarget formula produces them.

This explains the purpose of the metadata, but is not a complete validator for arbitrary retarget algorithms. The paper’s variable-difficulty argument imports a transition-rewriting lemma: an adversary whose malformed transitions escape some local tests can be replaced, without materially increasing its mining budget, by one respecting the model’s transitions and with the same sample-valid work. The lemma’s hypotheses include a sufficient aggregate-feasibility test. Our sampling theorem below assumes the resulting valid-weight bound; it does not independently prove that lemma, and does not instantiate its feasibility test for Zcash’s averaging-window retarget.

3 Choosing the sampling distribution

The real-number and continuous-probability construction in *Math Guide*, §*Probability and computation*, now gives an exact mathematical problem. Work is normalised to $[0, 1]$, and the final fraction δ is checked in full. The remaining samples are independent draws from a density on $[0, 1 - \delta]$. A density g assigns an interval $[a, b]$ probability $\int_a^b g(x) dx$; the integral is its area. The probability rules used here are those of *Math Guide*, §*Probability and computation*.

3.1 Why the tip receives more samples

A fork at a occupies weight $1 - a$. Under Assumption 1.2, at least $(1 - c)(1 - a)$ of this weight is invalid. Uniform sampling over the whole history catches it with probability only $(1 - c)(1 - a)$, which tends to zero as the fork moves towards the tip.

Choose an increasing density. For a fixed amount of invalid work after a , its least detectable placement is at the beginning of the fork, where the density is smallest. Moving any invalid mass from a later, higher-density region to an available earlier region cannot increase the catch probability. Repeating such moves leaves the interval

$$J_a = [a, 1 - c(1 - a)].$$

Its length is exactly $(1 - c)(1 - a)$. Extra invalid work only helps the verifier.

If this interval cannot fit before $1 - \delta$, then any placement avoiding the fully checked suffix is impossible. Otherwise the desired density should assign the same probability to J_a for every possible fork point.

3.2 Normalisation and the inverse sampler

Use the natural logarithm constructed in *Math Guide*, §Probability and computation: for $y > 0$, $\ln y = \int_1^y dt/t$. That construction proves the substitution rule, $\ln(ab) = \ln a + \ln b$, and the inverse exponential function. In particular, $\ln y$ is increasing, is zero at 1, and is negative for $0 < y < 1$.

Take $0 < \delta < 1$ and define

$$g_\delta(x) = \frac{1}{(1-x)\ln(1/\delta)} \quad (0 \leq x \leq 1-\delta).$$

The denominator is positive. Substituting $t = 1 - x$ gives

$$\int_0^z g_\delta(x) dx = \frac{-\ln(1-z)}{\ln(1/\delta)} =: G_\delta(z), \quad G_\delta(1-\delta) = 1.$$

Thus g_δ is a probability density. It increases towards the tip, but its support stops before the singularity at $x = 1$.

To draw a point, draw an independent uniform $U \in [0, 1]$ and solve $G_\delta(x) = U$:

$$x = 1 - \delta^U.$$

Indeed, G_δ is increasing, so $\Pr[1 - \delta^U \leq z] = G_\delta(z)$. A finite implementation must specify how its random bits approximate this ideal draw; floating-point rounding is not part of the mathematical construction.

For a block occupying $[a, b]$, its probability is the integral over that interval intersected with the sampling support. When $0 \leq a < b \leq 1 - \delta$, this is

$$p_{a,b} = \frac{\ln((1-a)/(1-b))}{\ln(1/\delta)}.$$

If the boundary cuts a block, fully checking that entire block is safe. It enlarges the fully checked region, never reducing its protection.

Theorem 3.1 (Single-sample detection). *Suppose a fork after a contains at least $(1-c)(1-a)$ invalid work. If no invalid work occurs in the fully checked suffix, a sample from g_δ catches invalid work with probability at least*

$$p = \frac{\ln(1/c)}{\ln(1/\delta)}.$$

The claim applies when $\delta \leq c$. If the invalid work cannot fit before the checked suffix, the suffix itself detects the fork.

Proof. By the increasing-density argument, the least detectable admissible placement is J_a . If it lies in the support, its mass is

$$\int_a^{1-c(1-a)} \frac{dx}{(1-x)\ln(1/\delta)} = \frac{\ln((1-a)/(c(1-a)))}{\ln(1/\delta)} = p.$$

If its right endpoint exceeds $1 - \delta$, there is insufficient uninspected work after a to contain all invalid work. Any admissible placement then intersects the suffix. $\square$

The proof works for whole blocks as well. Their invalid-work regions are finite unions of intervals, a restricted family of the placements allowed to the adversary in the fractional-work argument.

3.3 Optimality and its exact domain

The following optimality question asks for the best guarantee the verifier can achieve against the adversary's worst permitted placement. This best-against-worst criterion is called *minimax*.

Theorem 3.2 (Minimax optimality). *Let k be a positive integer and $\delta = c^k$. Among all one-sample densities supported on $[0, 1 - \delta]$, the largest guaranteed detection probability against the continuum fork model is $1/k$. The density g_δ attains it.*

Proof. For $i = 0, \dots, k - 1$, consider the fork point $a_i = 1 - c^i$. An admissible adversary makes exactly the interval

$$[1 - c^i, 1 - c^{i+1}]$$

invalid and places its valid fraction afterwards. These k intervals partition the sampling support, apart from endpoints of zero probability under a density. At least one interval receives probability at most $1/k$. Thus no density guarantees more than $1/k$. Theorem 3.1 gives $\ln(1/c)/\ln(1/c^k) = 1/k$ for g_δ . $\square$

The integer condition is used by the partition. For non-integral $k = \ln(1/\delta)/\ln(1/c) > 1$, the detection theorem still gives $1/k$, but this partition proof does not establish optimality. For $k = 1$, one sample suffices in the ideal model; expressions involving $\ln(1 - 1/k)$ are then undefined.

3.4 Choosing the sample count

After m independent draws, the probability of missing invalid work is at most

$$\epsilon_{\text{sample}} = (1 - p)^m.$$

To make this at most $2^{-\lambda}$, where $\lambda > 0$ is the desired error exponent, for $0 < p < 1$ choose

$$m = \left\lceil \frac{\lambda \ln 2}{-\ln(1 - p)} \right\rceil.$$

Taking logarithms preserves the inequality until division by the negative number $\ln(1 - p)$, when its direction reverses.

For constant difficulty, checking L of n history blocks gives $\delta = L/n$. Write $\log_b x = \ln x / \ln b$ for a logarithm to base $b > 0, b \neq 1$. For fixed c ,

$$p = \frac{\ln(1/c)}{\ln(n/L)}, \quad m = O(\lambda \log_{1/c}(n/L)).$$

The order bound follows from $-\ln(1 - p) \geq p$, which follows by integrating $1/(1 - t) \geq 1$ from zero to p . An authenticated sample costs one header and an $O(\log n)$ -node path. Without sharing duplicate paths, the proof therefore costs

$$O(L + \lambda \log_{1/c}(n/L) \log n)$$

header/node units, assuming bounded-size metadata. Actual bytes depend on header size and metadata encoding, not just this count.

Example 3.3 (A checked parameter calculation). Take $n = 2^{20}$, $L = 2^{10}$, $c = 1/2$, and $\lambda = 64$. Then $\delta = 2^{-10}$, $k = 10$, and $p = 1/10$. The least integer sample count is $m = 422$:

$$(9/10)^{422} \leq 2^{-64} < (9/10)^{421}.$$

This is an arithmetic example under the stated fork model, not a Zcash confirmation recommendation. The accompanying numerical check evaluates both inequalities with exact rational arithmetic.

4 The interactive verifier

The sampler is useful only when each challenge has a uniquely checkable answer. We can now state the complete verifier at the abstract history-interface level.

Construction 4.1 (Candidate verification). The verifier fixes the chain’s genesis, header rules, history commitment scheme, work function, suffix rule, c , and error budget.

1. Receive a candidate head, the authenticated history length and total work, and any enclosing commitment data needed to recover the history root. Check canonical encodings and the head’s own header validity. Freeze this candidate before choosing challenges.
2. Receive and validate the entire required suffix, including its ordering, previous-history roots, work and difficulty transitions. Check its connection to the head and authenticate its boundary in the committed history.
3. Draw m independent sample points. For each point obtain the selected header, its position, sibling metadata, and any adjacent header or epoch-boundary data required by the sample-validity rule.
4. Recompute the inclusion path and total weight. Check the work coordinate selects that leaf; check the header’s own proof of work, target and required difficulty data; recover and compare its prefix root as in Proposition 2.2.
5. Reject the candidate on any failed or missing answer. Otherwise retain it for work comparison with the other checked candidates.

Duplicate samples do not invalidate the bound: the draws remain independent even if two happen to select one header. A prover may compress repeated answers, but the verifier must retain the multiplicity of the original draws when analysing their distribution.

The empty-history and short-history cases use full verification rather than divide by zero or sample an empty support. A malformed proof is not a lower-work candidate. It is a rejected candidate. A timeout gives no positive assertion about chain validity.

Theorem 4.2 (Conditional interactive soundness). *Assume an honest candidate is available; the underlying chain has stable prefix security with failure probability $\varepsilon_{\text{chain}}$; the authenticated sample interface fails with probability at most $\varepsilon_{\text{auth}}$; and Assumption 1.2 holds with failure probability $\varepsilon_{\text{mine}}$. For one fixed competing candidate outside the short-fork exception, the probability that the verifier accepts it despite its invalid work is at most*

$$\varepsilon_{\text{auth}} + \varepsilon_{\text{mine}} + (1 - p)^m.$$

The stable-prefix conclusion additionally pays $\varepsilon_{\text{chain}}$.

Proof. Exclude the authentication and mining failure events. The frozen candidate then has well-defined work intervals and enough invalid work. The suffix either detects it, or Theorem 3.1 gives detection probability at least p for each independent challenge. All m challenges miss with probability at most $(1 - p)^m$. Adding back the excluded events gives the bound. Stable-prefix agreement with other honest nodes uses the separate consensus guarantee. $\square$

With several malicious candidates, apply a union bound using their actual number or a justified bound on it. The server must not be allowed to reset the experiment indefinitely under the same aggregate error budget. Availability of an honest proof and verification of maximal work are parts of the protocol, not consequences of this single-candidate theorem.

5 Fiat–Shamir and persistent verification

Interactive randomness prevents the server from tailoring a committed history to the sample. A transferable proof instead derives that randomness from the candidate. This removes interaction, but in-

troduces the possibility of trying many candidates.

5.1 Binding the challenge

The random-oracle model and transcript programming are defined in *Crypto Guide*, §*Security contracts and reductions* and §*Knowledge, simulation, and Schnorr*. Here the oracle supplies sampling randomness, not a Schnorr challenge or a witness extractor.

A non-interactive instantiation must hash a domain-separated, canonical description fixing the network, protocol version, candidate head, history root, history length, authenticated total work and sampling parameters. If some field is already unambiguously bound by another, its duplicate encoding is unnecessary; its binding is not.

The paper derives challenges from the candidate head. Under that construction, changing the samples entails changing and mining a head. Nevertheless, the cost of trying heads is a mining-model question. It is not enough to say that a hash looks random.

Proposition 5.1 (A finite grinding budget). *Suppose each fresh candidate has conditional sample-evasion probability at most ϵ given the transcript before its fresh oracle answer. An adversary makes at most Q such attempts. Its probability of finding an evading candidate is at most $Q\epsilon$.*

Proof. For each attempt, its success event has probability at most ϵ , by averaging the conditional bound. The probability of the union of these events is at most their sum. The candidates need not be chosen independently. $\square$

Consequently a $2^{-\lambda}$ aggregate sampling target needs

$$(1 - p)^m \leq 2^{-\lambda}/Q.$$

The choice Q must cover all permitted restarts and candidate-grinding opportunities. The paper gives a bound in its own mining model; it is not a universal limit on an implementation’s hash queries.

Finite sampling precision creates another explicit error term. Compare the *selected-header indices*, a finite distribution, with the indices induced by the ideal continuous draw. A finite set of numerical coordinates is not close in statistical distance to a continuous distribution on real coordinates: those supports differ completely. If each implemented index distribution is within statistical distance η of its ideal index distribution, independently pair the implemented and ideal draws so that each pair differs with probability at most η . This pairing is called a coupling; its finite construction is given in *Math Guide*, §*Probability and computation*. The probability that any of the m pairs differs is at most $m\eta$, by the union bound. The security budget must include that term, or use a separately proved exact discrete index sampler.

5.2 State retained by a returning client

A returning client can retain a stable authenticated checkpoint rather than begin again from genesis. A new candidate must prove consistency with that checkpoint and satisfy the sampling requirements for its extension. Old accepted work does not have to be retransmitted merely because a new suffix arrived.

A current head is not necessarily stable. A client may replace its unsettled suffix while retaining the earlier common prefix; it must not turn a previously tentative observation into irrevocable finality. The appropriate confirmation rule remains the one in *Consensus Guide*, §*Reorganisation and confirmation confidence*.

After accepting the header history, the client can request a block’s membership proof and then a transaction’s membership proof under that block’s transaction commitment. Effecting-data and authorising-data commitments must be checked separately where the transaction format separates them. A transaction-inclusion response alone must not be treated as evidence of fresh chain selection or transaction execution.

6 Connecting the construction to Zcash

Zcash's history commitment is already part of the core consensus construction. The *Consensus Guide*, §*Authenticated block history*, specifies its canonical metadata, its one-block delay, upgrade-epoch resets, and the outer block commitment. These are inputs to a possible client verifier, not an implementation of the verifier above.

The distinction has concrete consequences:

1. A Zcash history tree covers one upgrade epoch. A proof spanning upgrades must authenticate the transitions and recover the relevant earlier roots. It cannot interpret the latest root as a genesis-rooted tree.
2. The head commits to earlier blocks, not to itself. Its own work, validity and total-score contribution must be checked separately.
3. The root uses Zcash's bagging and node encoding. Paper proofs must be translated and verified under those exact rules.
4. The root is nested inside the block commitment alongside authorising-data commitments. The verifier needs the appropriate enclosing values to reconstruct the header's committed digest.
5. Zcash's retarget rule is not the paper's long, clamped-epoch model. The aggregate-feasibility predicate and a justified valid-fork-weight bound remain necessary work.
6. The wire protocol must specify requests, proofs, integer limits, canonical random-bit sampling, error handling, checkpoint persistence and multi-server comparison. A node's ability to compute a root does not provide those interfaces automatically.

ZIP 221, in its security discussion, explicitly leaves a stronger analysis for rapidly adjusting chains before a FlyClient ecosystem is designed. This is a real boundary of the selected construction, not a historical footnote. The light-client trust contract examined in *Sync Guide*, §*Server trust and observable metadata*, does not become trustless because a block contains a history root.

The result of this volume is therefore precise. The sampling density, its normalisation, its integer-case optimality and its error accounting are complete mathematical constructions. Authenticated prefix and weight checks state the exact interface they consume. Instantiating their security premises and proof transport for Zcash is not completed by ZIP 221, by this text, or by the existence of a valid history commitment.